

System Programming Project 4

담당 교수 : 김영재 교수님

이름 : 김동건

학번 : 20211507

1. 개발 목표

이번 프로젝트에서는 C 프로그램의 dynamic memory allocator를 작성한다. malloc, free, realloc을 직접 설계하고, 제공하는 mdriver를 통해 효율성을 체크해보며 최적의 dynamic memory allocator를 만드는 것이 목표이다.

2. 개발 내용

강의 시간에 배운 여러 방식의 dynamic memory allocator가 존재하지만, 이번 프로젝트에서는 segregated free list를 이용하는 방식으로 구현하였다. 이는 기본적으로 implicit, explicit list보다 효율적으로 작동한다.

전체적인 큰 그림을 설명하면 다음과 같다. segregated free list의 각 리스트에서는 free 블록을 내림차순으로 관리하였다. 이를 위해 insert_node 과정에서 맨 앞 혹은 맨 끝에 넣는 것이 아닌 탐색을 통해 내림차순을 유지하는 위치에 노드를 삽입하였다. 이렇게 관리하였기 때문에 메모리를 할당받을 때는 각 구간에 맞는 segregated free list의 가장 앞에서부터 순회하며 가장 여유가 작은 메모리 블록을 찾아 할당하였고, 그러지 못한 경우에는 mem_sbrk를 통해 메모리를 추가로 할당해 주었다. free 이후에는 앞 뒤로 merge할 수 있는지 확인 후, merge를 진행한다. realloc의 경우, 이전 realloc 사이즈와의 차이를 여분으로 추가 할당하는 식으로 구현하여 메모리 사용 효율을 높였다. 차이만큼 조금 여유있게 블록을 할당하여 다음에 추가로 힘을 할당하지 않게 하여 mem_sbrk를 최소한으로 호출하게 구현하였다.

이제, 코드를 보며 살펴보기 전, allocated block과 free block의 구조를 살펴보자. 이를 이해하여야 아래 설명할 매크로, 코드들을 이해할 수 있다. 먼저 allocated block은 [header / payload / footer] 구조이다. header/footer는 크기가 32비트 즉, 4바이트며 하위 3개의 비트를 제외한 비트에 블록의 사이즈를 저장한다. 최하위 비트에는 allocated 여부를 저장한다. (즉, 1일 것이다.) 반면 free block은 [header / prev 포인터 / next 포인터 / ... / footer] 구조이다. ...으로 한 이유는 관리를 하고 있는 free 블록은 크기가 다양하기 때문이다. 따라서 ... 구간은 free block마다 크기가 다르다. header, footer는 allocated block과 같은 구조이다. 다만, header 바로 뒤에 포인터를 저장하는 바이트가 2개 있는데, 이는 free list에서 앞 블록, 뒷 블록의 주소를 저장하는 공간이다. 보고서의 3번 항목에서 시각적으로 확인하면 이해에 도움이 될 것이다.

매크로를 살펴보자. 이는 강의 교재 csapp의 figure 9.43을 참고하였다. 다만, 본인

스타일대로 조금의 수정을 가한 부분도 있다.

```
// csapp figure 9.43 매크로
#define WSIZE 4           // 워드, 헤더, 푸터 사이즈
#define DSIZE 8           // 더블 워드 사이즈
#define SSIZE 3           // 헤더/푸터에서 status 나타내는 비트 범위 [0, 3)
#define MAX_PW 32         // segregated 범위 2^MAX_PW
#define CHUNKSIZE (1 << 12) // 힘을 이 양만큼 확장 (4096)

#define MAX(x, y) ((x) > (y) ? (x) : y) // max 함수

#define PACK(size, alloc) ((size << SSIZE) | alloc) // 사이즈랑 할당된비트를 한 워드로 묶는 거

// 주소 p에 있는 거 읽기, 쓰기
#define GET(p) (*(unsigned int *)(p))
#define PUT(p, val) (*(unsigned int *)(p) = (val))

// size랑 allocate 정보 가져오기
#define GET_SIZE(p) ((GET(p) & ~0x7) >> SSIZE) // ~0x7 : 111..111000. 페이로드 사이즈
#define GET_ALLOC(p) (GET(p) & 0x1) // 0x1: 000...000001

// 헤더 포인터로 푸터 주소 계산 (payload + 헤더 사이즈 더하면 나옴)
#define FTRP(bp) ((char **)(bp) + GET_SIZE(bp) + 1)

#define NEXT_BLK(p) (FTRP(bp) + 1) // 다음 블록 헤더 주소
#define PREV_BLK(p) ((char **)(bp) - GET_SIZE((char **)(bp) - 1) - 2) // 앞 블록 헤더 주소 (2만큼 더 빼야함.)

#define PREV_NODE(p) (*(char **)(bp + 1)) // 이전 노드 포인터
#define NEXT_NODE(p) (*(char **)(bp + 2)) // 다음 노드 포인터
```

차이점은 GET_SIZE인데, 강의자료와 달리 저장하고 싶은 payload 값을 3번째비트부터 밀어서 저장하였다. 즉, GET_SIZE와 PACK부분이 강의자료와 다르게 3비트 밀어서 사이즈를 관리함에 유의하여야 한다. 또, PREV_BLK의 정의도 조금 상이하며, free node에서 PREV_NODE, NEXT_NODE를 반환하는 매크로도 작성하여 코드의 가독성을 높이려고 하였다.

```
static char **segregated_free_list; // 다중 free list 관리용 포인터
int prev_size; // realloc 전 사이즈 저장용. 메모리 realloc 할 때마다
```

segregated_free_list는 다중 list를 관리하기 위한 포인터 주소다. prev_size는 앞에서 언급한 이전 realloc 사이즈 사용하기 위해 관리하는 변수이다. 이제, 추가한 함수들을 살펴보자. mm_init, mm_malloc, mm_realloc, mm_free 외에 새로 추가한 함수는 아래와 같다.

```
static void insert_node(char **ptr);
static void remove_node(char **ptr);
static void place(void *ptr, size_t words);
static void *merge(void *ptr);
static void mark_block(char **ptr, size_t size, int alloc);
static int get_index(size_t size);
```

프로그램의 호출 순서를 감안하여 위 사진과 다른 순서로 함수를 살펴보겠다. 우선 mark_block 함수이다.

```
// free, alloc 표시 하는 거. size만큼  
static void mark_block(char **ptr, size_t size, int alloc) { PUT(ptr, PACK(size, alloc)), PUT(ptr + size + 1, PACK(size, alloc)); }
```

어떤 블록에 free, alloc 여부를 표시하는 역할이다. 중복된 코드가 너무 많아 이렇게 함수로 따로 작성하였다.

다음은, get_index 함수이다. 이는 어떤 블록의 사이즈가 주어졌을 때, segregated의 몇 번째 리스트에 속하는지 찾아주는 함수이다.

```
// 인덱스 찾기용  
static int get_index(size_t size) {  
    int idx = 0, tmp = 1;  
    while ((tmp << 1) <= size) idx++, tmp <<= 1;  
    return idx;  
}
```

2의 제곱수를 계산해가며 인덱스를 찾는다.

다음은 merge함수이다. free된 블록이 앞뒤로 연속되어 있는 게 있으면 합쳐주는 함수이다.

```

// free 앞 뒤 보고 merge 하기
static void *merge(void *ptr) {
    char **pre_ptr = PREV_BLKPTR(ptr), **suf_ptr = NEXT_BLKPTR(ptr);
    size_t pre_alloc = GET_ALLOC(pre_ptr), suf_alloc = GET_ALLOC(suf_ptr);
    size_t size = GET_SIZE(ptr);

    if (pre_alloc && suf_alloc) return ptr; // 둘 다 차있으면 못 합침
    if (pre_alloc && !suf_alloc) {          // 뒤랑 합치기
        remove_node(suf_ptr);
        size += GET_SIZE(suf_ptr) + 2;
    }
    else if (!pre_alloc && suf_alloc) {     // 앞랑 합치기
        remove_node(pre_ptr);
        size += GET_SIZE(pre_ptr) + 2;
        ptr = pre_ptr;
    }
    else {                                  // 앞 뒤로 합치기
        remove_node(pre_ptr), remove_node(suf_ptr); // 삭제
        size += GET_SIZE(pre_ptr) + GET_SIZE(suf_ptr) + 4;
        ptr = pre_ptr;
    }
    mark_block(ptr, size, 0);
    return ptr;
}

```

강의에서 배운 대로 4가지 경우에 대해 블록을 merge해 주었다.

다음은 remove_node이다. 이는 free 리스트에서 노드를 삭제해주는 함수이다.

```

// segregated_free_list에서 ptr이 가리키는 노드 삭제
static void remove_node(char **ptr) {
    if (ptr == NULL) return;

    size_t size = GET_SIZE(ptr);
    if (size == 0) return; // 이거 없으면 더미헤더땀에 문제 생김... 하...

    char **pre_ptr = PREV_NODE(ptr), **suf_ptr = NEXT_NODE(ptr);

    // 앞
    if (pre_ptr) NEXT_NODE(pre_ptr) = suf_ptr; // pre의 뒤 노드는 지금의 Suf
    else segregated_free_list[get_index(size)] = suf_ptr; // 맨앞인 경우 강 뒤예거를 앞에 붙이기만

    // 뒤
    if (suf_ptr) PREV_NODE(suf_ptr) = pre_ptr; // suf 노드의 앞 노드는 지금의 pre
}

```

삭제한 노드의 앞 뒤를 잘 이어준다.

다음은 insert_node로, segregated free list에 삽입하는 함수이다.

```
// free list에 추가하기
static void insert_node(char **ptr) {
    if (!ptr) return;
    size_t size = GET_SIZE(ptr);
    if (size == 0) return; // 이거 없으면 더미헤더땀에 문제 생김... 하...

    int idx = get_index(size);
    PREV_NODE(ptr) = NULL, NEXT_NODE(ptr) = NULL; // 앞 뒤 다 NULL로 초기화

    if (!segregated_free_list[idx]) segregated_free_list[idx] = ptr; // 비어 있으면 바로 넣고 끝.
    else if (size >= GET_SIZE(segregated_free_list[idx])) { // 맨 앞..
        NEXT_NODE(ptr) = segregated_free_list[idx], PREV_NODE((char **)segregated_free_list[idx]) = ptr;
        segregated_free_list[idx] = ptr;
    }
    else {
        char **pre_ptr = segregated_free_list[idx];
        while (NEXT_NODE(pre_ptr) != NULL && GET_SIZE(NEXT_NODE(pre_ptr)) > size) pre_ptr = NEXT_NODE(pre_ptr);
        char **suf_ptr = NEXT_NODE(pre_ptr);
        NEXT_NODE(pre_ptr) = ptr, PREV_NODE(ptr) = pre_ptr; // 앞 노드에 내 정보 추가, 내 거에 앞 정보 추가
        if (suf_ptr) NEXT_NODE(ptr) = suf_ptr, PREV_NODE(suf_ptr) = ptr; // 내 거에 뒤 정보 추가, 뒤에 내 정보 추가
    }
}
```

삽입할 인덱스를 찾고, 빈 경우, 맨 앞에 삽입해야 하는 경우, 나머지 경우로 나누어 노드를 추가한다.

마지막으로 place 함수이다. 해당 위치에 payload만큼을 할당하고, 남는 것은 free_list에 추가해주는 함수이다. malloc과 realloc에서 호출된다. 이때, payload를 ptr에 무조건 할당할 수 있는 경우에만 호출하였음에 유의해야 한다. 관련하여 개발자의 실수가 있을 수 있기에 assert문을 넣어주었다.

```
// ptr에 payload 할당
// 호출 때 가능하게 잘 해야 함... (assert 참고)
static void place(void *ptr, size_t payload) {
    size_t want_size = payload + 2, original_size = GET_SIZE(ptr) + 2;
    assert(original_size >= want_size); // 할당 가능한 경우에만 호출했는데, 혹시 모르니 assert로 확인,,

    if (original_size - want_size < 2) mark_block(ptr, original_size - 2, 1); // 2도 안남으면 free 역할 못함.
    else { // 2넘게 남으면 쪼개서 추가.
        mark_block(ptr, payload, 1); // 새로 할 거 1로 마킹
        mark_block((char **)ptr + want_size, original_size - (want_size + 2), 0); // 남는 거 0으로 마킹
        insert_node((char **)ptr + want_size); // 남는 거 free list에 추가
    }
}
```

한 가지 더 주의할 점으로는 남는 공간이 2 이하이면 payload를 넣을 공간이 없으니 free 블록으로 관리할 가치가 없어 그냥 할당만 하고 끝낸다는 것이다.

이제 위 매크로, 함수, 변수들을 가지고 과제에서 요구하는 mm_init, mm_malloc, mm_realloc, mm_free 함수들을 하나씩 살펴보자.

먼저, mm_init이다.

```
/*
 * mm_init - initialize the malloc package.
 * 오류: -1, else: 0 리턴
 */
int mm_init(void) {
    prev_size = 0;

    char **bp = mem_sbrk(MAX_PW * sizeof(char *));

    // segregated free list 개수 만큼 할당
    if ((segregated_free_list = bp) == (void *)-1) return -1;

    // NULL로 초기화
    for (int i = 0; i < MAX_PW; i++) segregated_free_list[i] = NULL;

    // alignment 용.
    // 오히려 없어야 할 거 같은데.. 빼면 오류가 남.. π
    bp = mem_sbrk(WSIZE);
    if (bp == (void *)-1) return -1;

    // heap 할당 4개 (더미 head, foot 각 2개씩)
    bp = mem_sbrk(4 * WSIZE);
    if (bp == (void *)-1) return -1;

    // heap 시작 끝에 각각 size 0인 더미 블록(head/foot이고 사이즈 0인) 뒤서 표시
    for (int cnt = 0; cnt < 2; cnt++, bp += DSIZE) mark_block(bp, 0, 1);

    return 0;
}
```

우선, realloc에서 사용할 prev_size 변수를 초기화해주고, segregated를 관리하는 데에 필요한 메모리들을 할당하여 준다. 또, heap의 앞 뒤로 더미 노드를 두어 heap의 끝임을 알 수 있게 관리하므로 그것을 위한 메모리도 할당하고 사이즈가 0인 블록으로 마킹하였다. 이때, alignment를 신경써야 함에 유의하자.

```

/*
 * mm_malloc - Allocate a block by incrementing the brk pointer.
 *             Always allocate a block whose size is a multiple of the alignment.
 */
void *mm_malloc(size_t size) {
    if (!size) return NULL;
    size_t rsz = size;

    if (size <= CHUNKSIZE) {
        rsz = 1;
        while (rsz < size) rsz <= 1; // 나 이상 첫 2 제곱 수
    }

    // align으로 8바이트 맞추고, 워드로 나누기. 필요한 워드 수
    size_t need = (ALIGN(rsz)) / WSIZE;

    char **bp = NULL;
    int idx = get_index(need);
    while (idx < MAX_PW) {
        char **free_list_node = segregated_free_list[idx++];
        if (free_list_node == NULL || GET_SIZE(free_list_node) < need) continue;
        while (NEXT_NODE(free_list_node) && GET_SIZE(NEXT_NODE(free_list_node)) >= need) free_list_node = NEXT_NODE(free_list_node);
        bp = free_list_node, idx = MAX_PW;
    }

    if (!bp) {
        size_t cnt = MAX(need, CHUNKSIZE / WSIZE); // 늘릴 워드 개수
        bp = mem_sbrk((cnt + 2) * WSIZE); // 헤더 푸터 포함 2개 더
        if (bp == (void *)-1) return NULL;

        // 새로 만든 거 넣고, 맨 끝 더미 밀기
        // 기존 끝 더미 노드 2칸 빼야 함..
        // alloc은 여기서 안 함. 0으로

        // payload
        mark_block(bp - 2, cnt, 0);

        // 아래 2개가 힙 끝 더미
        mark_block(bp + cnt, 0, 1);

        bp = bp - 2;
    }
    else remove_node(bp); // 찾았으면 그 블록은 seg list에서 삭제

    // 넣고, 남은 거 다시 할당
    place(bp, need);
    return bp + 1; // payload라 1 더해서
}

```

다음은 mm_malloc이다. 우선 예외처리 이후에, 필요한 워드 수를 계산한다. 다음으로, free_list에 해당 크기를 할당할 수 있는 블록이 있는지 확인한다. 있다면 else로 가서 그 블록을 삭제한 후, place로 여유가 있다면 자르고 할당을 해준다. 만약 없다면 mem_sbrk로 새 메모리를 받아서 할당하고, place로 여유가 있다면 자르고 할당을 해준다.


```

/*
 * mm_free - Freeing a block does nothing.
 */
void mm_free(void *ptr) {
    if (!ptr) return;
    ptr -= WSIZE; // payload 시작 주소에서 헤더 주소로 변경

    size_t pay_load = GET_SIZE(ptr); // 페이로드 크기 (헤더, 푸터 뺀 거)
    mark_block(ptr, pay_load, 0);

    // 합친 다음에 segmented에 free 노드추가
    insert_node(merge(ptr));
}

```

다음은 mm_free이고, 페이로드 크기만큼 0으로 변경한다. merge로 앞 뒤 free가 있다면 합친 뒤, free_list에 추가한다.

마지막은 mm_realloc이다. 가장 신경을 많이 쓴 부분이다.

```

/*
 * mm_realloc - Implemented simply in terms of mm_malloc and mm_free
 */
void *mm_realloc(void *ptr, size_t size) {
    if (ptr == NULL) return mm_malloc(size); // 없으면 malloc
    if (size == 0) { // 0 이면 free
        mm_free(ptr);
        return NULL;
    }
    // 여기까지 명세서 언급된 예외 1, 2 처리

    int dif = ((int)(size - prev_size) > 0 ? (int)(size - prev_size) : (int)(prev_size - size));
    prev_size = size;

    int buf_size = ((size + 500) / 1000) * 1000; // 1000 반올림..
    size_t rdif = 1;
    if (dif <= CHUNKSIZE) { // 근데 작으면 2제곱수 가깝게,,
        while (rdif < dif) rdif <<= 1;
        if (dif != rdif) buf_size = rdif;
    }

    char **original_ptr = (char **)ptr, **bp = original_ptr - 1; // 원래 거

    size_t cur = ALIGN(size) / WSIZE; // 버퍼 땀 필요한 거
    size_t need = cur + buf_size; // 필요한 거
    size_t original_size = GET_SIZE(bp); // 원래 사이즈

    if (original_size >= cur) return ptr; // 이미 여유 있으면 걍 그거. 굳이 버퍼 L..

    char **pre_ptr = PREV_BLKPTR(bp), **suf_ptr = NEXT_BLKPTR(bp);
    size_t pre_size = GET_SIZE(pre_ptr), suf_size = GET_SIZE(suf_ptr);
    int pre_alloc = GET_ALLOC(pre_ptr), suf_alloc = GET_ALLOC(suf_ptr);

```

```

    if (pre_alloc && !suf_alloc && suf_size + original_size + 2 >= need) { // 뒤에 합치기
        mark_block(bp, original_size, 0);
        bp = merge(bp);
        place(bp, need);
    }
    else if (!pre_alloc && suf_alloc && pre_size + original_size + 2 >= need) { // 앞에 합치기
        mark_block(bp, original_size, 0);
        bp = merge(bp);
        memmove(bp + 1, original_ptr, original_size * WSIZE);
        place(bp, need);
    }
    else if (!pre_alloc && !suf_alloc && pre_size + suf_size + original_size + 2 >= need) {
        mark_block(bp, original_size, 0);
        bp = merge(bp);
        memmove(bp + 1, original_ptr, original_size * WSIZE);
        place(bp, need);
    }
    else { // 새로 할당 받기
        char **new_bp_header = (char **)mm_malloc((need * WSIZE) + WSIZE) - 1;
        if (!new_bp_header) return NULL;
        memcpy(new_bp_header + 1, original_ptr, original_size * WSIZE);
        mm_free(original_ptr);
        return new_bp_header + 1;
    }
    return bp + 1;
}

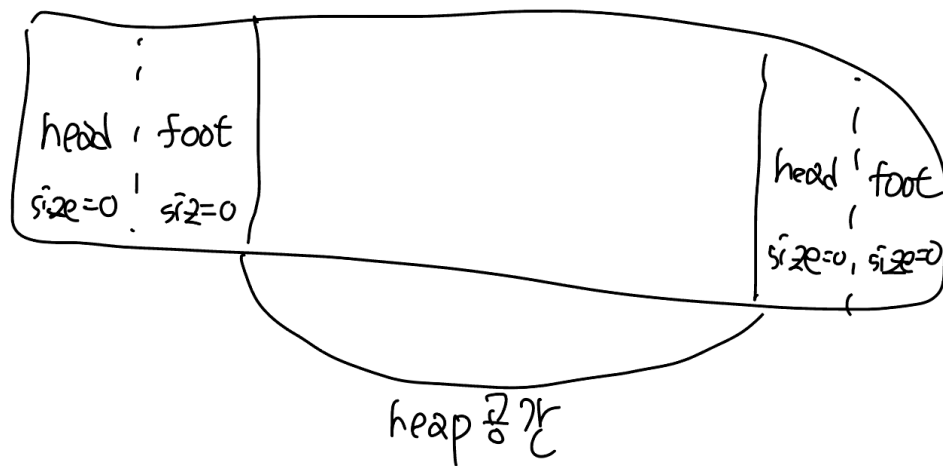
```

우선, 명세서에서 언급한 경우에 대해 예외 처리를 하였다. 다음으로 이전 realloc과 차이를 통해 다가올 realloc 명령에 대한 예상을 하였고, 그 값만큼 여유있게 할당을 하는식으로 구현하였다. 그 과정에서 chunksize보다 작으면 2제곱수에 가깝게 여유를 잡았고, 그 이상이라면 1000단위로 반올림하여 여유를 잡았다.

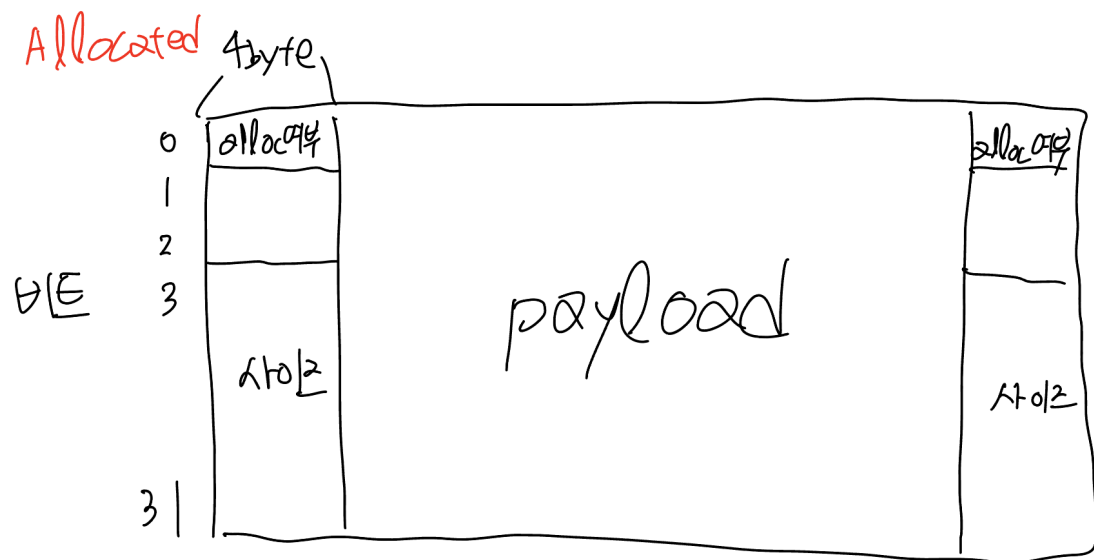
그 후, 이미 기본 크기에 여유가 있다면 아무 작업이 필요 없으므로 해당 포인터를 바로 리턴하였고, 부족한 경우는 앞에서 끌어쓰기, 뒤에서 끌어쓰기, 앞 뒤에서 끌어쓰기를 순서대로 보며 합쳐서 가능하면 합쳐서 할당해 주었다. 모두 불가능하다면 mm_malloc을 통해 추가로 메모리를 할당받아 사용하였다.

3. 참고 시각 자료

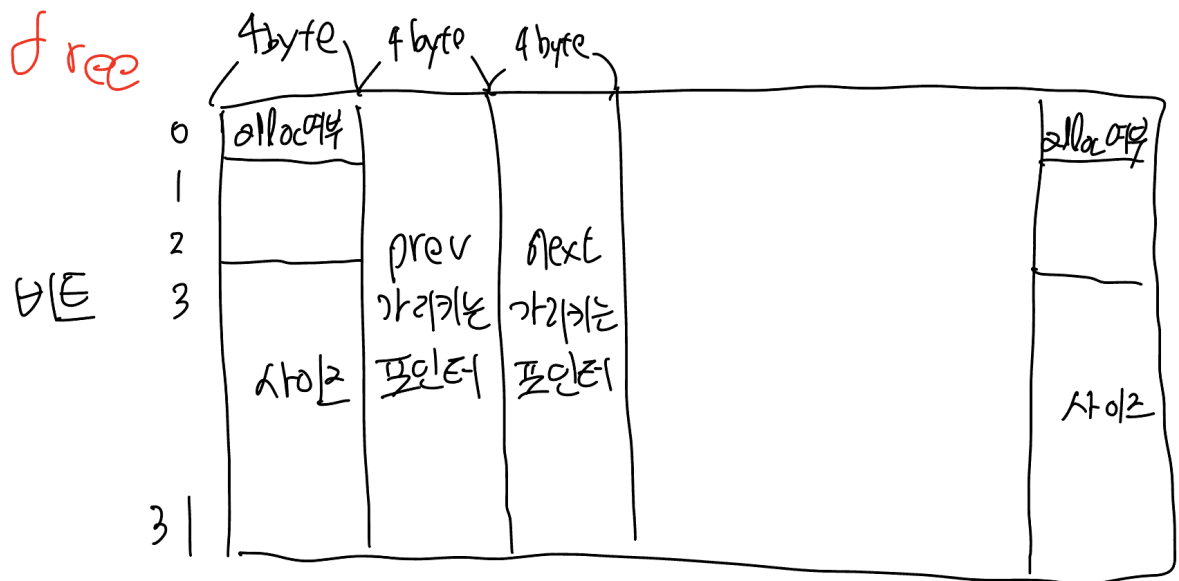
아래 그림들은 위에서 설명한 자료구조들을 시각적으로 나타낸 것이다.



위 그림은 mm_init에서 언급한 heap 공간 앞뒤에 있는 더미노드를 시각화 한 것이다.



위 그림은 allocated block의 구조이다. 앞 뒤 4byte(32bit)는 헤더와 푸터이고, 제일 낮은 비트에 alloc 여부를 저장하고, 3번칸부터는 사이즈를 저장한다.



위 그림은 free block의 구조이다. allocated와 비슷하지만, 헤더 뒤에 prev, next free block을 가리키는 포인터를 추가로 저장한다.

4. 결론

강의 교재에 나와 있는 implicit 방식으로 구현했을 때보다 segregated로 수정한 뒤 점수가 상승하였다. 이는 free list들을 크기에 나눠 log 베이스로 관리하여 효율성을 얻었기 때문이다. 위 구현으로 97점을 획득할 수 있었고, 데이터를 확인해보니 64, 448이 반복해서 들어오고 사이 블록만 계속 해제하여 단편화를 극대화하는 데이터 등이 있었는데, 해당 데이터들을 저장하여 더 높은 점수를 얻을 수 있었다. 그러나 이러한 데이터 저장 방식은 미리 메모리 요청을 알 수 없는 실제 상황과는 괴리가 있다고 판단하여 반영하지 않았다.